🛡️ THREAT INTELLIGENCE PIPELINE & SIEM DETECTION FRAMEWORK

End-to-End Cybersecurity Implementation

Cybersecurity | Detection Engineering | SIEM | Python Automation

---

by Miguel Kent

Project Overview

This project demonstrates the design and implementation of an end-to-end threat intelligence pipeline leveraging Python automation and the Splunk platform for detection, analysis, and response.

A virtualized lab environment was built using Ubuntu Server within Oracle VirtualBox, providing a controlled setting for simulating real-world security operations. Python was utilized to automate the collection of threat intelligence data by sending HTTP requests to publicly available feeds and retrieving structured JSON datasets containing known malicious IP addresses.

The collected data was processed through a normalization phase, where key attributes such as IP address, port, malware classification, and timestamp were extracted and standardized into a consistent schema. This ensured compatibility with SIEM ingestion and enabled efficient querying and correlation.

Secure file transfer between the virtual machine and host system was enabled using SSH, allowing for efficient movement of processed datasets into the SIEM environment for ingestion.

The normalized dataset was then ingested into Splunk, where it was indexed and transformed into searchable events. Detection engineering techniques were applied to correlate threat intelligence indicators with simulated network activity, allowing for the identification of malicious connections based on known indicators of compromise.

To operationalize detection, scheduled alerts were configured within Splunk to automatically trigger when matches were identified. A dashboard was developed to provide visibility into detection activity, including trends, frequency of threat indicators, and overall event volume.

Threat intelligence enrichment was performed using geolocation capabilities to add contextual data such as country of origin, enhancing the analytical value of detections and improving situational awareness.

Finally, an incident response workflow was implemented to simulate real-world analyst processes, including alert validation, activity investigation, impact assessment, and response actions such as monitoring and blocking malicious IPs.

This project demonstrates the integration of automation, secure data handling, SIEM-based detection, and incident response to create a complete and operational threat intelligence-driven security workflow.

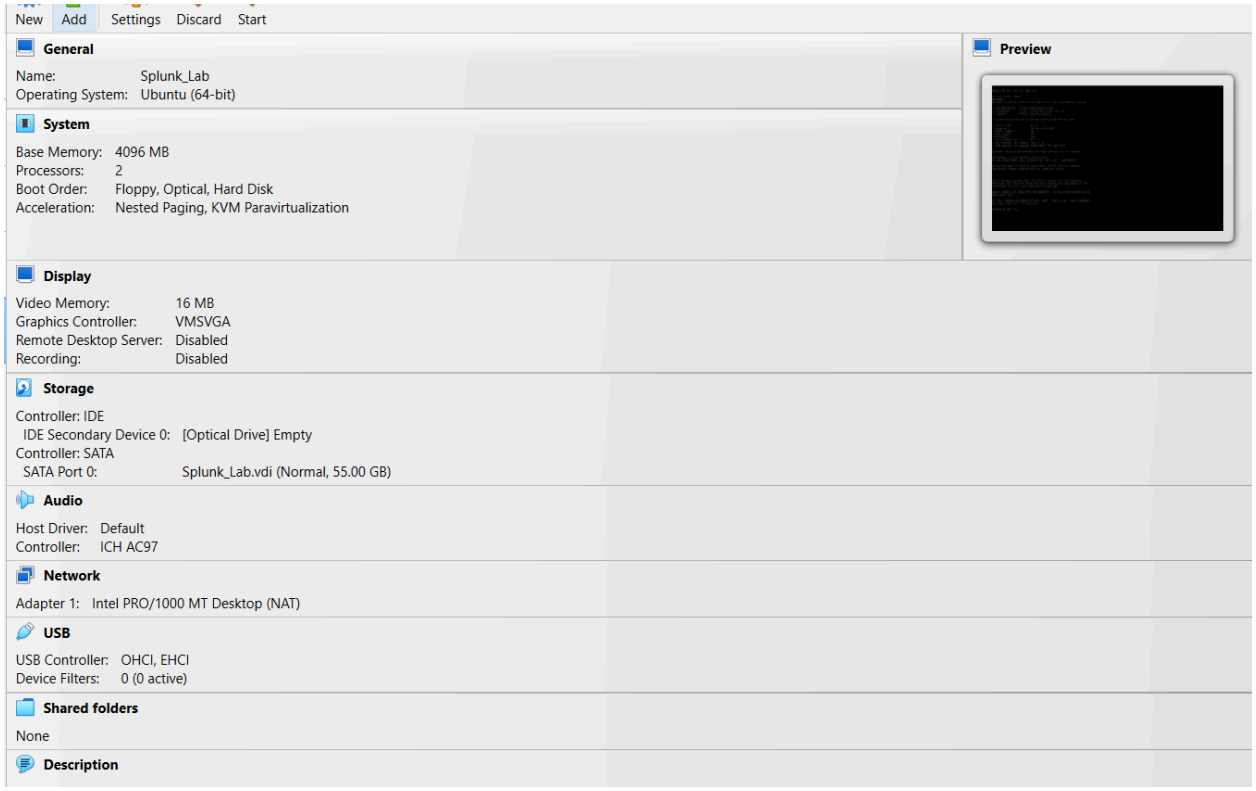
1. Environment Setup

A virtualized lab environment was created to support the deployment of a threat intelligence pipeline and SIEM platform. The system was built using Ubuntu Server 24.04 within Oracle VirtualBox, providing a lightweight and controlled environment aligned with real-world server deployments.

Configuration:

- Platform: Oracle VirtualBox
- Operating System: Ubuntu Server 24.04
- Memory: 4 GB RAM
- CPU: 2 vCPUs
- Storage: 50 GB (LVM)
- Hostname: `cti-lab`
- Remote Access: OpenSSH enabled

This environment serves as the foundation for all subsequent phases, including SIEM deployment, threat intelligence ingestion, and detection engineering.



2. SIEM Deployment (Splunk)

A Security Information and Event Management (SIEM) platform was deployed within the lab environment using Splunk Enterprise. The platform was installed directly on the Ubuntu Server system to enable centralized data ingestion, analysis, and detection capabilities.

Prior to installation, the system was updated to ensure compatibility and stability of all required dependencies. The Splunk installation package was then retrieved from the official distribution source using command-line tools and extracted within the server environment.

Following extraction, Splunk was initialized through the command-line interface. During this process, licensing terms were accepted and administrative credentials were configured, enabling secure access to the platform. The service was successfully started and configured to run locally on the host system.

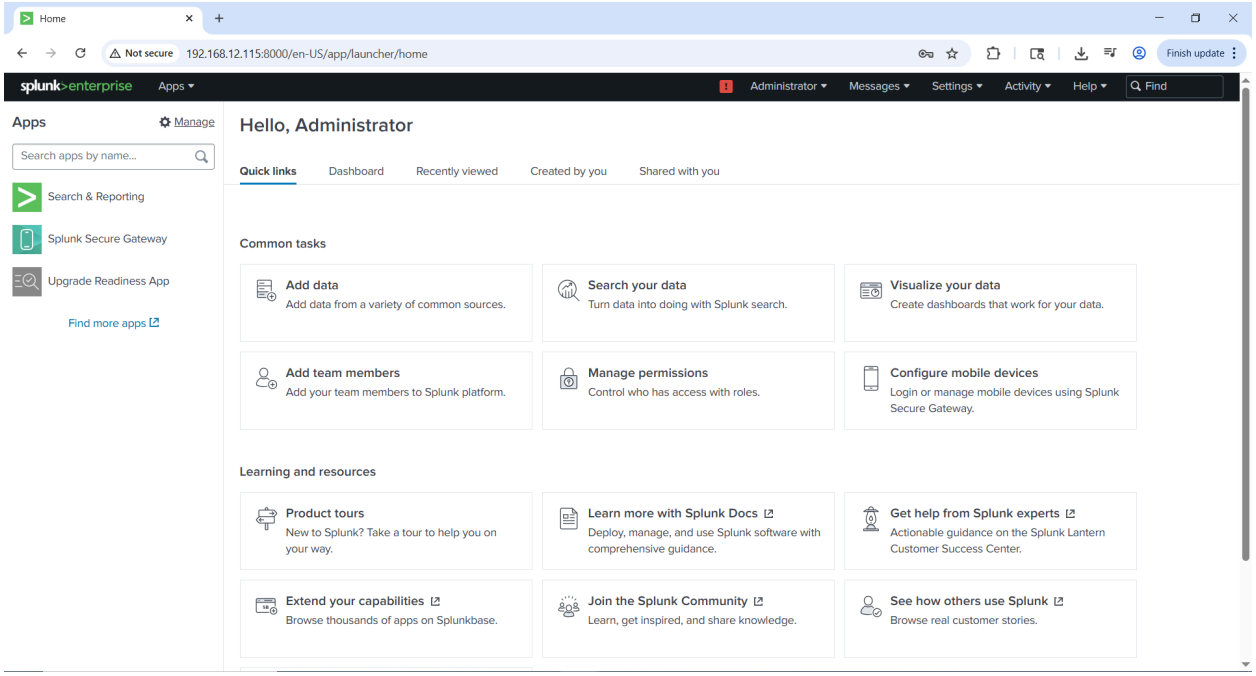
To facilitate access to the SIEM interface, network settings were adjusted within the virtualized environment, allowing the Splunk web interface to be accessed via browser over port 8000. Upon successful configuration, the platform was accessed through its web interface, confirming full operational status.

This deployment establishes the core monitoring platform required for ingesting and analyzing threat intelligence data, forming the central component of the overall threat detection pipeline.

```
mkent@cti-lab:~$ wget -O splunk.tgz "https://download.splunk.com/products/splunk/releases/9.1.2/linux/splunk-9.1.2-b6b9c8185839-Linux-x86_64.tgz"
--2026-03-29 02:08:00--  https://download.splunk.com/products/splunk/releases/9.1.2/linux/splunk-9.1.2-b6b9c8185839-Linux-x86_64.tgz
Resolving download.splunk.com (download.splunk.com)... 99.84.234.35, 99.84.234.48, 99.84.234.87, ...
Connecting to download.splunk.com (download.splunk.com)|99.84.234.35|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 614861791 (586M) [binary/octet-stream]
Saving to: ‘splunk.tgz’

splunk.tgz                          100%[===================================================================================>] 586.38M  4.48MB/s    in 1m 56s

2026-03-29 02:09:56 (5.07 MB/s) - ‘splunk.tgz’ saved [614861791/614861791]

mkent@cti-lab:~$ _
```

```
splunk/etc/modules/input/structuredparsing/
splunk/etc/modules/input/structuredparsing/config.xml
splunk/etc/modules/input/TCP/
splunk/etc/modules/input/TCP/config.xml
splunk/etc/modules/input/fschangemanager/
splunk/etc/modules/input/fschangemanager/config.xml
splunk/etc/modules/output/
splunk/etc/modules/output/config.xml
splunk/etc/modules/output/RemoteQueue/
splunk/etc/modules/output/RemoteQueue/config.xml
splunk/etc/modules/distributedDeployment/
splunk/etc/modules/distributedDeployment/classes/
splunk/etc/modules/distributedDeployment/classes/deployable/
splunk/etc/modules/distributedDeployment/classes/deployable/inputs.conf
splunk/etc/modules/distributedDeployment/classes/deployable/outputs.conf
splunk/etc/modules/distributedDeployment/classes/deployable/README
splunk/etc/modules/distributedDeployment/classes/deploymentserver/
splunk/etc/modules/distributedDeployment/classes/deploymentserver/deployment.conf
splunk/etc/shcluster/
splunk/etc/shcluster/apps/
splunk/etc/shcluster/apps/README
splunk/etc/shcluster/users/
splunk/etc/shcluster/users/README
splunk/etc/log-utility.cfg
splunk/etc/prettyprint.xsl
splunk/etc/anonymizer/
splunk/etc/anonymizer/anonymizer-time.ini
splunk/etc/anonymizer/private-terms.txt
splunk/etc/anonymizer/names.txt
splunk/etc/anonymizer/public-terms.txt
splunk/etc/anonymizer/dictionary.txt
splunk/etc/init.d/
splunk/etc/init.d/README
splunk/etc/master-apps/
splunk/etc/master-apps/_cluster/
splunk/etc/master-apps/_cluster/local/
splunk/etc/master-apps/_cluster/local/README
splunk/etc/master-apps/_cluster/default/
splunk/etc/master-apps/_cluster/default/indexes.conf
splunk/etc/splunk-enttrial.lic
splunk/etc/splunk-launch.conf.default
splunk/etc/findlogs.ini
splunk/etc/log-cmdline.cfg
splunk/etc/deployment-apps/
splunk/etc/deployment-apps/README
splunk/etc/searchLanguage.xml
splunk/etc/log-debug.cfg
mkent@cti-lab:~$ ls
splunk  splunk.tgz
mkent@cti-lab:~$ _
```

```
        Checking configuration... Done.
                Creating: /home/mkent/splunk/var/lib/splunk
                Creating: /home/mkent/splunk/var/run/splunk
                Creating: /home/mkent/splunk/var/run/splunk/appserver/i18n
                Creating: /home/mkent/splunk/var/run/splunk/appserver/modules/static/css
                Creating: /home/mkent/splunk/var/run/splunk/upload
                Creating: /home/mkent/splunk/var/run/splunk/search_telemetry
                Creating: /home/mkent/splunk/var/run/splunk/search_log
                Creating: /home/mkent/splunk/var/spool/splunk
                Creating: /home/mkent/splunk/var/spool/dirmoncache
                Creating: /home/mkent/splunk/var/lib/splunk/authDb
                Creating: /home/mkent/splunk/var/lib/splunk/hashDb
New certs have been generated in '/home/mkent/splunk/etc/auth'.
        Checking critical directories...        Done
        Checking indexes...
                Validated: _audit _configtracker _internal _introspection _metrics _metrics_rollup _telemetry _thefishbucket history main summary
        Done
        Checking filesystem compatibility...  Done
        Checking conf files for problems...
        Done
        Checking default conf files for edits...
        Validating installed files against hashes from '/home/mkent/splunk/splunk-9.1.2-b6b9c8185839-linux-2.6-x86_64-manifest'
        All installed files intact.
        Done
All preliminary checks passed.

Starting splunk server daemon (splunkd)...
Generating a RSA private key
.............+++++
.............+++++
writing new private key to 'privKeySecure.pem'
-----
Signature ok
subject=/CN=cti-lab/O=SplunkUser
Getting CA Private Key
writing RSA key
PYTHONHTTPSVERIFY is set to 0 in splunk-launch.conf disabling certificate validation for the httplib and urllib libraries shipped with the embedded Python inter
preter; must be set to "1" for increased security
Done


Waiting for web server at http://127.0.0.1:8000 to be available..... Done


If you get stuck, we're here to help.
Look for answers here: http://docs.splunk.com

The Splunk web interface is at http://cti-lab:8000

mkent@cti-lab:~/splunk/bin$ _
```



3. Threat Intelligence Collection

To establish the foundation of the threat intelligence pipeline, external threat data was integrated into the lab environment using Python. This phase focused on retrieving real-world indicators of compromise (IOCs) from publicly available intelligence sources.

A Python-based approach was implemented using the `requests` library to enable communication with external threat feeds over HTTP. Initial testing validated API connectivity and response handling, ensuring the system could successfully interact with external intelligence sources.

Following successful connectivity testing, a public threat intelligence dataset was utilized to retrieve structured data containing known malicious IP addresses. The selected feed provided JSON-formatted data, allowing for efficient parsing and processing within the Python environment.

The script was designed to send a request to the external feed, validate the response, and convert the returned data into a usable format. A subset of the data was then displayed to confirm successful ingestion and to verify the structure of the retrieved indicators.

This phase demonstrates the ability to ingest real-time threat intelligence from external sources, forming the basis for subsequent processing, normalization, and integration into the SIEM platform for detection and analysis.

```
if response.status_code == 200:
        data  = response.json()

        cleaned_data =[]

        for entry in data :
            cleaned_entry = {
                "ip": entry.get("ip_address"),
                "port": entry.get("port"),
                "malware": entry.get("malware"),
                "timestamp": entry.get("first _seen")
            }
            cleaned_data.append(cleaned_entry)

        with open ("cleaned threat_intel.json","w") as f:
            json.dump(cleaned_data, f, indent=4)

        print("Cleaned data saved:", len(cleaned_data), "entries")

else:
        print("Failed to retreive data")
```

```
mkent@cti-lab:~$ python threat_intel.py
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
mkent@cti-lab:~$ python3 threat_intel.py
Cleaned data saved: 5 entries
mkent@cti-lab:~$ ls
'cleaned threat_intel.json'   cti-env   splunk   splunk.tgz   threat_intel.py
mkent@cti-lab:~$
```

4. Data Normalization

Following the successful retrieval of external threat intelligence, the collected data was processed and normalized to ensure consistency and usability within a SIEM environment. Raw threat feeds often contain varying structures and extraneous data, making direct ingestion inefficient for detection and correlation.

To address this, a Python-based normalization process was implemented to extract and standardize key fields from each threat record. Relevant attributes such as IP address, associated port, malware classification, and first-seen timestamp were selectively parsed from the dataset.

Each record was transformed into a consistent schema, ensuring uniformity across all entries regardless of the original data format. This structured approach enables more effective indexing, searching, and correlation within downstream systems.

The normalized data was then written to a JSON file, creating a clean and portable dataset suitable for ingestion into the SIEM platform. This step establishes a critical bridge between raw intelligence collection and actionable detection, ensuring that threat indicators can be efficiently leveraged for monitoring and analysis.

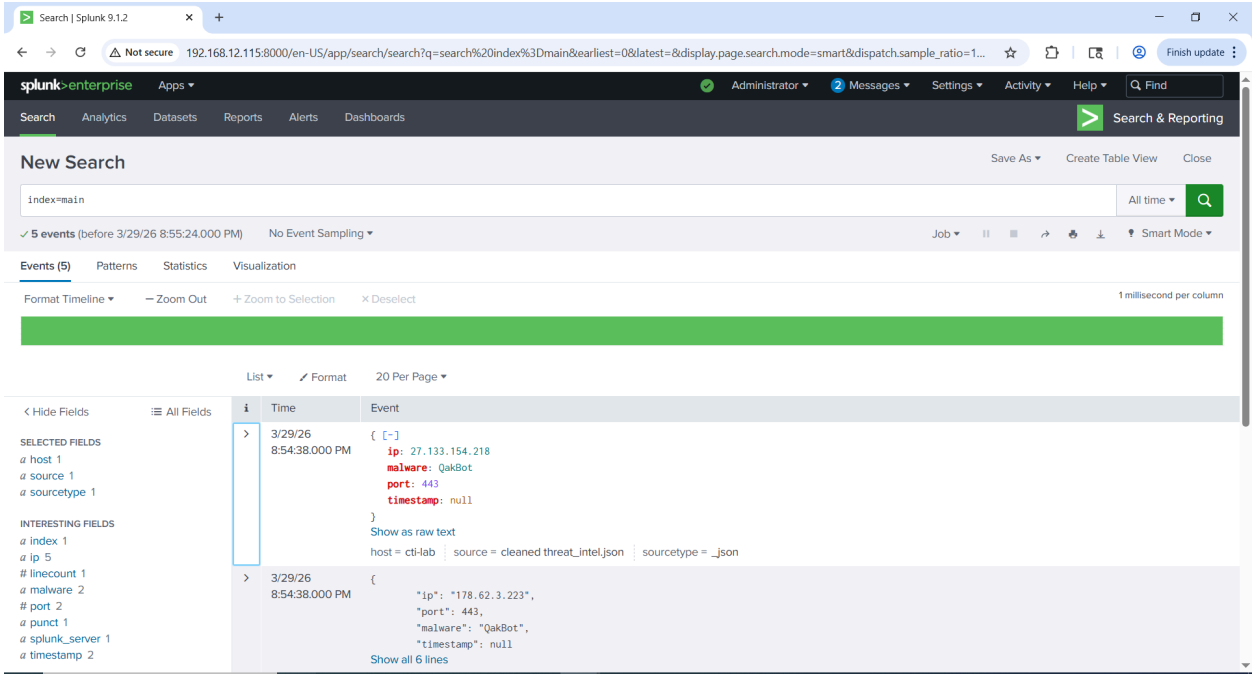
```
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd00::a00:27ff:fee1:5cf1


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


mkent@cti-lab:~$ cd ~
mkent@cti-lab:~$ ls
'cleaned threat_intel.json'   cti-env   fake_log.log   splunk   splunk.tgz   threat_intel.py
mkent@cti-lab:~$ cat cleaned\threat_intel.json
cat: cleanedthreat_intel.json: No such file or directory
mkent@cti-lab:~$ cat cleaned\ threat_intel.json
[
    {
        "ip": "162.243.103.246",
        "port": 8080,
        "malware": "Emotet",
        "timestamp": null
    },
    {
        "ip": "50.16.16.211",
        "port": 443,
        "malware": "QakBot",
        "timestamp": null
    },
    {
        "ip": "34.204.119.63",
        "port": 443,
        "malware": "QakBot",
        "timestamp": null
    },
    {
        "ip": "178.62.3.223",
        "port": 443,
        "malware": "QakBot",
        "timestamp": null
    },
    {
        "ip": "27.133.154.218",
        "port": 443,
        "malware": "QakBot",
        "timestamp": null
    }
]mkent@cti-lab:~$ _
```

5. SIEM Data Ingestion

Following the normalization of threat intelligence data, the structured dataset was ingested into the SIEM platform to enable analysis and correlation. The cleaned JSON file containing malicious IP indicators was transferred from the Linux-based lab environment to the host system and uploaded into Splunk.

Within Splunk, the dataset was indexed and made searchable through the platform's Search and Reporting interface. This step validated that the threat intelligence data was successfully integrated into the SIEM, allowing for visibility into the imported indicators and confirming proper data ingestion.

By centralizing the processed threat intelligence within Splunk, the environment is now capable of leveraging these indicators for detection and analysis. This phase establishes the connection between external intelligence and internal monitoring, enabling the development of detection logic based on known malicious activity.



6. Detection Engineering — Threat Intelligence Correlation

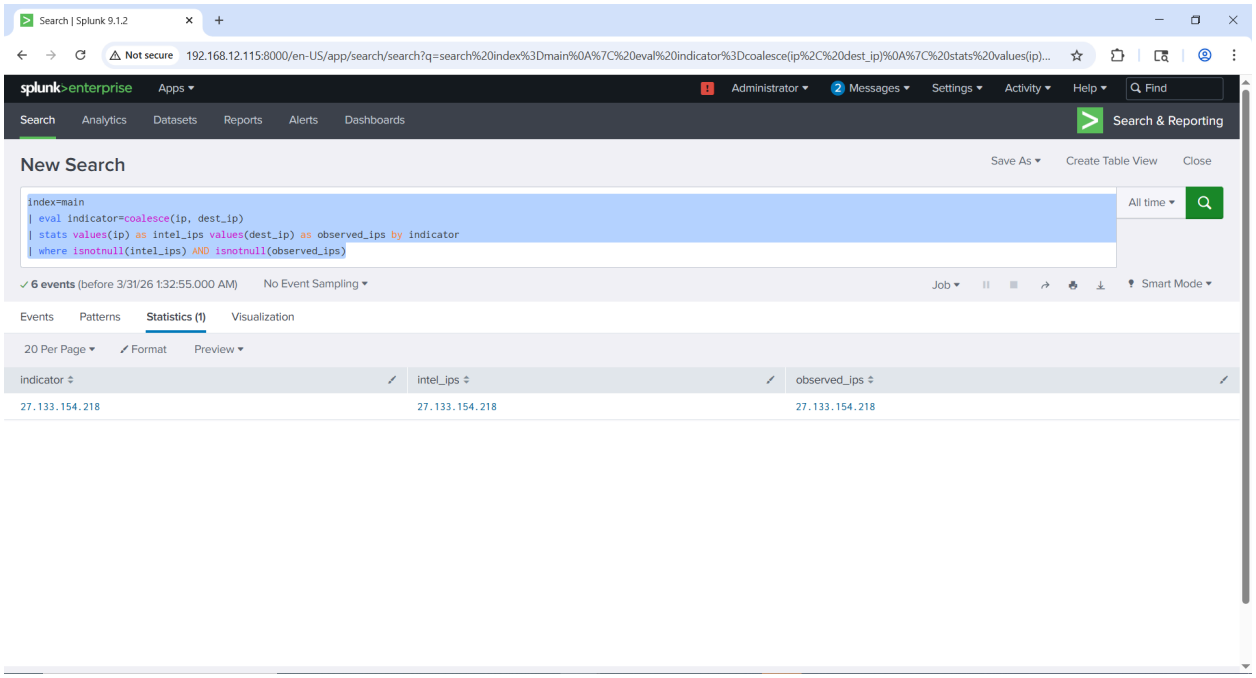
Following the successful ingestion of both threat intelligence data and simulated log activity, detection logic was developed to identify potential security events through correlation.

The objective of this phase was to determine whether observed system activity matched any known malicious indicators contained within the threat intelligence dataset. To achieve this, a correlation search was implemented within Splunk, comparing destination IP addresses from simulated logs against known malicious IPs.

Since the threat intelligence data and simulated logs were ingested as separate event types, a normalization step within the search logic was required to align the relevant fields. This was accomplished by combining the fields into a unified comparison structure, enabling effective matching across datasets.

The detection logic identified instances where an observed destination IP matched a known malicious indicator, successfully simulating a real-world detection scenario in which internal systems communicate with potentially harmful external infrastructure.

This phase demonstrates the practical application of threat intelligence by transforming static indicator data into actionable detection logic. By correlating external intelligence with internal activity, the system is capable of identifying high-risk events and supporting proactive security monitoring.



7. Alerting — Operationalizing Detection

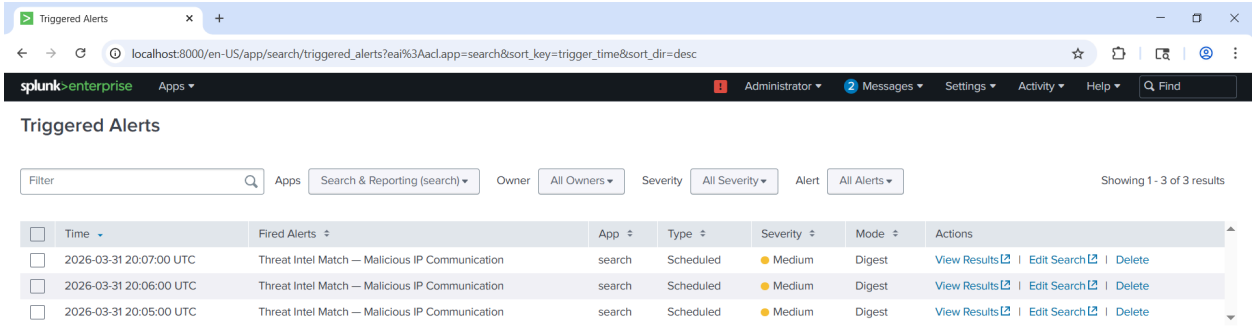
Following the development of correlation-based detection logic, the final step was to operationalize this detection by configuring an automated alert within Splunk.

The previously developed search query, which identifies matches between observed network activity and known malicious IP indicators, was converted into a scheduled alert. This enables continuous monitoring without requiring manual execution of the detection logic.

The alert was configured to run at regular intervals and trigger whenever matching results are identified. A trigger condition was defined to activate the alert when the number of results exceeds zero, indicating that at least one observed event corresponds to a known malicious indicator.

Upon execution, the alert successfully triggered based on the simulated log activity, confirming that the detection pipeline is functioning as intended. The alert was recorded within Splunk's alerting interface, providing visibility into the detected event and establishing a mechanism for ongoing monitoring.

This phase completes the transition from passive data analysis to active threat detection, demonstrating how threat intelligence can be operationalized to automatically identify and respond to potential security risks.



8. Dashboard Visualization and Monitoring

Following the implementation of detection and alerting mechanisms, a dashboard was developed within Splunk to provide a centralized and visual representation of threat intelligence activity.

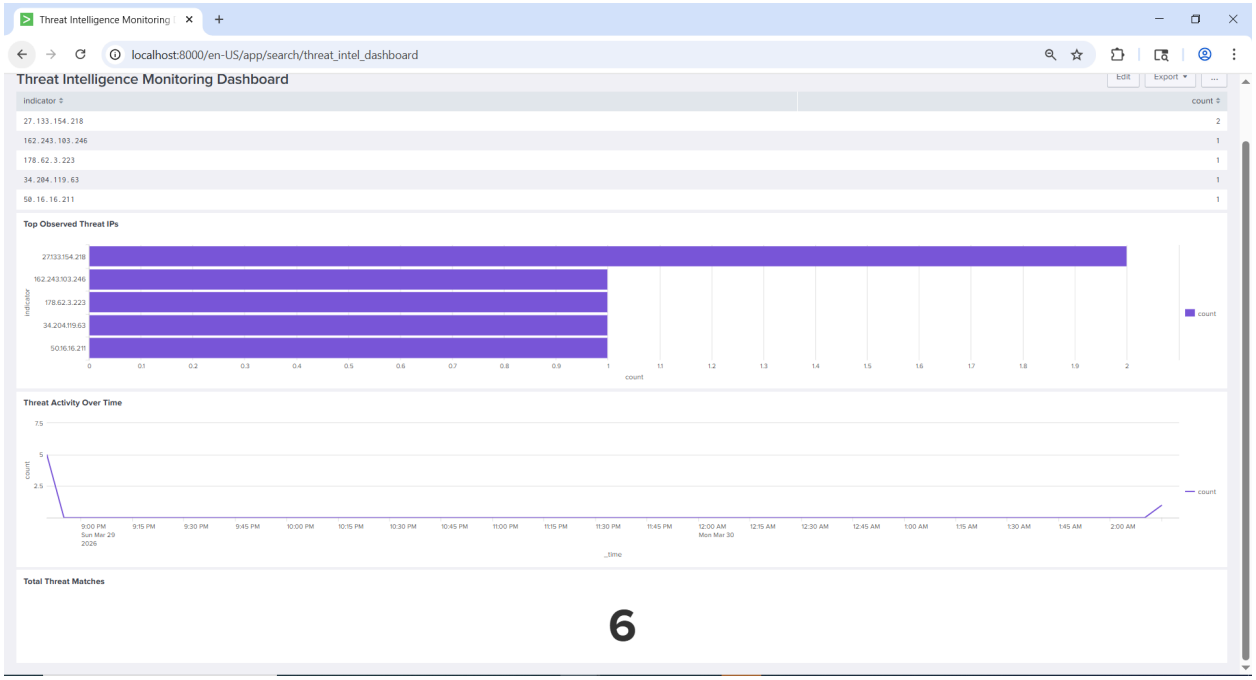
The purpose of this phase was to enhance visibility into detected events by transforming raw search results into intuitive visualizations. This enables more efficient monitoring and supports both technical analysis and high-level reporting.

The dashboard was designed to display key metrics related to threat intelligence correlation. Multiple panels were created to present different perspectives of the data, including the frequency of observed malicious IP addresses, the progression of activity over time, and the overall volume of detected events.

A bar chart was implemented to highlight the most frequently observed threat indicators, allowing for quick identification of high-risk IP addresses. A time-based visualization was used to track activity trends, providing insight into when detections occur and enabling the identification of potential spikes or anomalies. Additionally, a single-value panel was included to summarize the total number of threat matches, offering a clear and immediate understanding of detection volume.

During implementation, it was identified that default time constraints limited data visibility within the dashboard. This was resolved by adjusting the search parameters to include all available data, ensuring accurate and complete representation of the dataset.

This phase introduces a monitoring layer to the threat intelligence pipeline, allowing for continuous observation of detection activity. By presenting data in a structured and visual format, the system becomes more accessible to analysts and stakeholders, bridging the gap between raw data and actionable insight.



9. Threat Intelligence Enrichment

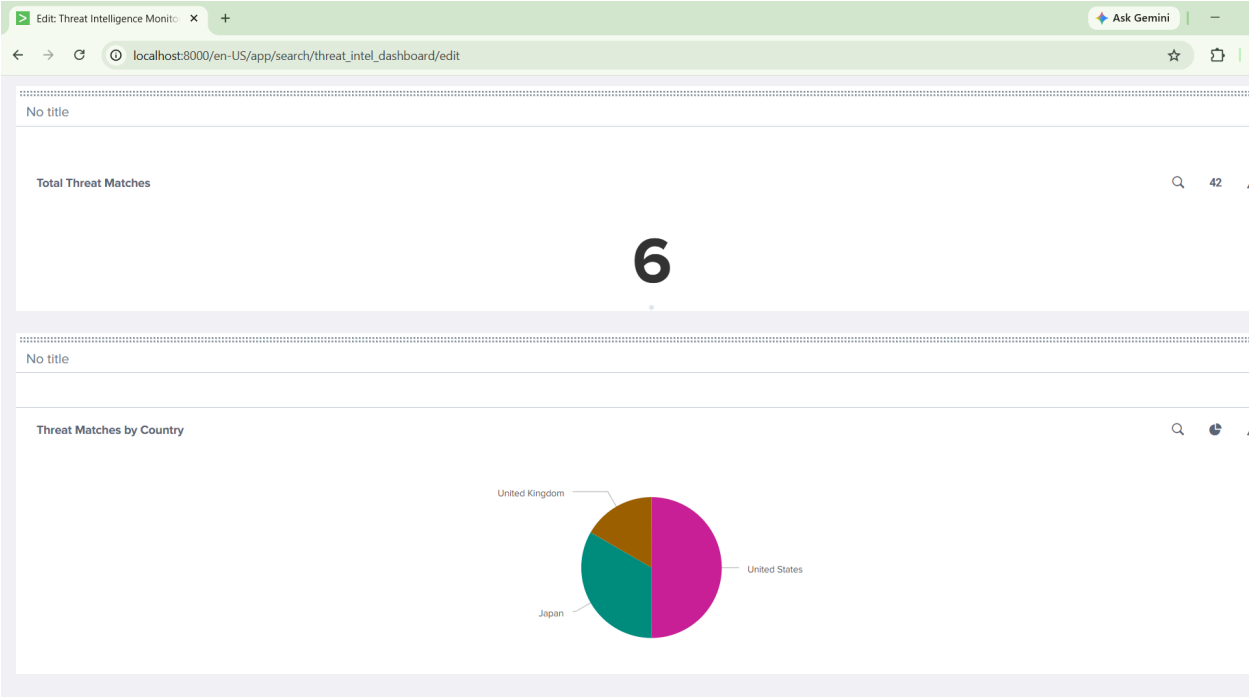
Following the implementation of detection and visualization, threat intelligence enrichment was introduced to provide additional context to identified indicators. While initial detections confirmed matches between observed activity and known malicious IP addresses, these indicators lacked meaningful context for deeper analysis.

To address this, enrichment was performed within Splunk using built-in geolocation capabilities. The `iplocation` function was applied to the correlated IP addresses, augmenting each indicator with geographic data such as country of origin.

This enhancement allowed for the transformation of basic detection data into more informative intelligence. By associating IP addresses with geographic locations, it became possible to identify patterns, understand the distribution of threat activity, and provide additional insight during analysis.

The enriched data was then visualized to highlight the distribution of threat activity by country. This provided a clear and intuitive view of where detected malicious communications were originating, further improving situational awareness.

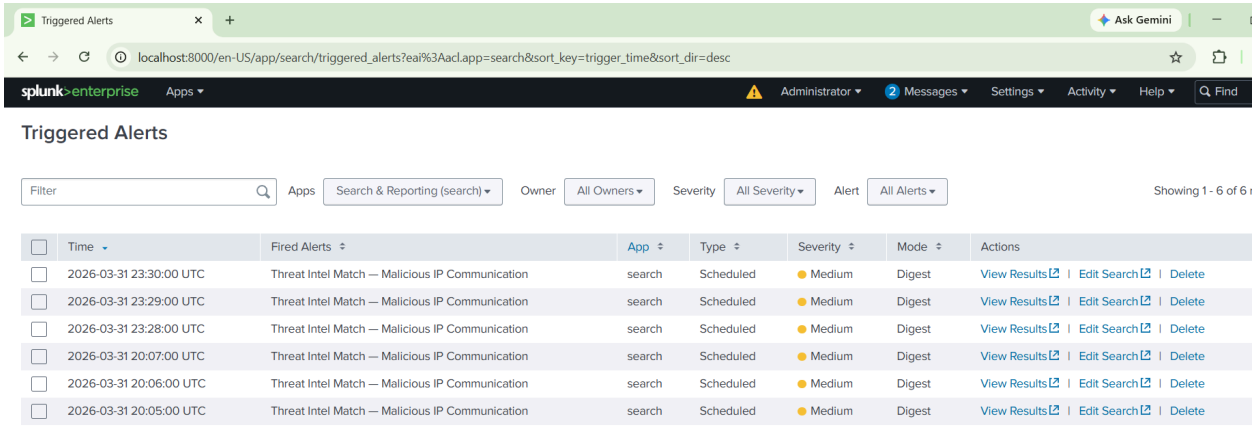
This phase demonstrates the importance of context in cybersecurity operations. Enrichment bridges the gap between raw indicators and actionable intelligence, enabling more effective investigation and decision-making.



10. Incident Response and Investigation

Following the successful detection and alerting of malicious activity, an incident response workflow was conducted to investigate and assess the identified event.

The process began by reviewing the triggered alert within Splunk to confirm the details of the detection. The alert indicated that an internal system had communicated with a known malicious IP address identified through the threat intelligence dataset.

Further analysis was performed by examining the associated event data, including the source and destination IP addresses. The internal source system was then investigated by querying additional activity related to the originating IP to determine whether the behavior was isolated or part of a broader pattern.

The analysis revealed that the activity was limited and consistent with a controlled simulation, indicating low impact and no evidence of persistent or widespread compromise. Based on this assessment, appropriate response actions were identified, including monitoring the affected system and blocking the malicious IP to prevent future communication.